

Agent Foundry — Reference Architecture

Agent Foundry

September 3, 2026

CONTENTS

- Agent Foundry
 - A Reference Architecture for Building Any LLM Agent
 - 1. What This Is
 - 2. Design Principles
 - 3. The Stack
 - 3.1 Layer 01 — UI/UX
 - 3.2 Layer 02 — Orchestration
 - 3.3 Layer 03 — Harness / Runtime Plane
 - 3.4 Layer 04 — Tools Gateway
 - 3.5 Layer 05 — LLM Gateway
 - 3.6 Layer 06 — Context Layer
 - 3.7 Left Rail — Identity & Governance
 - Autonomy levels
 - 3.8 Right Rail — Evaluation, Guardrails & Reinforcement
 - Evaluation — four levels
 - The KPI framework — comprehensive, and pluggable
 - Guardrails — four gates
 - Planning
 - Reinforcement loop
 - Operational tooling — flags, versions, experiments, batch
 - 4. Security, Observability & Cost

5. The Extensibility Model

6. Deployment

7. Module Reference

Agent Foundry

A Reference Architecture for Building Any LLM Agent

Version 0.4 — 2026

1. What This Is

Agent Foundry is a modular framework for building LLM agents — support bots, sales assistants, research agents, operations copilots, anything — on a single reusable core. Every layer is a small contract (a Python `Protocol`), not a mandate. A team wires in what a given agent needs and leaves the rest at its default.

It is built on **LangGraph** for orchestration and, where a lighter on-ramp matters more than control, on real **LangChain** abstractions (`langchain.agents.create_agent`, native structured tool-calling) rather than a bespoke convention.

The organizing idea is a strict split between two kinds of code:

Central — everything in the `agent_foundry/` package. Shared infrastructure, built once, identical whether a team runs one agent or fifty. Nothing in it names a specific agent, industry, or task.

Specific — everything about *one* agent: its prompt (a plain text file), the tools it has been given, its policy (what it may do, spend, and when it needs a human), and which orchestration topology fits its job. This is the entire surface a new agent touches.

A new agent is a prompt file, a short list of tools, and one configuration object. Everything else — routing, permissions, guardrails, evaluation, cost tracking, audit — is infrastructure a team writes once and never touches again per agent.

Every claim in this document was verified against running code during development: real MCP servers, a real OPA policy server, a real HashiCorp Vault instance, a real vector database, a real Slack signature, live HTTP round trips. Where something is a design intention rather than a tested behavior, it is marked as such.

2. Design Principles

Modular by default. Six layers, two governance rails, and a set of cross-cutting pillars (security, observability, cost, evaluation). Fill in what the use case needs; leave the rest dark.

Protocol-driven, not inheritance-driven. Nearly every extension point in the framework is a Python `Protocol` — a structural contract with one or two methods. A replacement doesn't need to subclass anything; it needs to match the shape. This is what makes “bring your own LLM / tool / vector database / guardrail engine / planning algorithm” a real, load-bearing property of the system rather than a slogan.

Composable, not hardcoded. Orchestration is not one fixed agent loop. It is three primitives — `make_think_node`, `make_act_node`, `make_router` — that seven different topologies are built from. A DAG step can call a supervisor graph as a tool. A swarm agent can itself be a debate panel.

Provider-agnostic. No layer hardcodes a cloud, a model vendor, a vector database, or a messaging platform. The container is the only portability boundary; nothing inside `agent_foundry/` imports a cloud-specific SDK.

Evaluate every layer, not just the final answer. Four levels of evaluation (atomic, component, flow, overall) plus a KPI framework for scoring anything a team cares about, in whatever combination and weighting fits.

Human oversight where it's expensive to be wrong. A graduated, six-level autonomy model — not a binary approve/deny — decides how much a given action can happen without a person.

3. The Stack

Requests flow down from the surface a human or system touches, through orchestration and runtime, out to tools and models, and land on a context layer that remembers. An identity and governance rail runs alongside every layer; an evaluation, guardrail, and reinforcement rail watches every layer from the other side.

3.1 Layer 01 — UI/UX

Where humans meet the agent.

- **Conversational surface** — a CLI REPL (`ui/console.py`) that drives any compiled graph
- **HITL console** — the same module handles approval prompts inline, resuming a paused graph via LangGraph's `Command(resume=...)`
- **Dashboard renderer** — a text summary of cost, eval scores, and steps for one thread
- **Channels** (`channels.py`) — connects external messaging surfaces to a compiled graph. `build_slack_app()` is a full, tested Slack Events API integration: real HMAC-SHA256 request signing, the URL-verification handshake, and message events routed through the graph to a posted reply
- **HTTP serving** (`serve.py`) — wraps any compiled graph as a minimal REST API (`POST /chat` , `GET /health`), the deployment surface a load balancer or container orchestrator talks to

Not built, and out of scope for this repository: a web-based Agent Studio, a voice interface, and mobile SDKs. These are separate frontend products, not backend framework code.

3.2 Layer 02 — Orchestration

The LangGraph engine, and the layer most people mean when they say “agent framework.”

Two primitives do all the work:

- `make_think_node(config)` — one LLM call: reads the system prompt (optionally augmented with retrieved context), calls the model, applies output guardrails, returns the response
- `make_act_node(config)` — one tool call: checks the circuit breaker, checks action guardrails (including the autonomy-level gate), invokes the tool, records the result
- `make_router(config)` — decides whether the next step is another tool call or the end of the turn; records flow-level evaluation and closes the cost ledger the moment a task completes

Every orchestration topology in the framework is these three primitives, wired differently:

Topology	Function	What it's for
Single Agent	<code>build_agent_graph</code>	One agent, a think/act loop
Supervisor	<code>build_supervisor_graph</code>	A central LLM router dispatches each turn to one of several named specialists, via LangGraph's <code>Command(goto=...)</code>
Swarm / Handoff	<code>build_swarm_graph</code>	Decentralized — any specialist can hand off directly to a peer with no central router
Fan-out	<code>build_fanout_graph</code>	One agent applied to many inputs in parallel, via LangGraph's <code>Send</code> primitive (true map-reduce)
Blackboard	<code>build_blackboard_graph</code>	Multiple agents read and write to a shared workspace (Facts, Hypotheses, Evidence, Tasks, Contradictions, Open Questions) instead of talking to each other directly
Debate / Judge	<code>build_debate_graph</code>	N agents answer independently; a judge agent reviews all answers and synthesizes a final one
DAG Workflow	<code>build_dag_graph</code>	A fixed, non-agentic pipeline of deterministic steps with dependencies — for work that needs reliability, not judgment, at every hop
Agents as Tools	<code>agent_as_tool</code>	Wraps any compiled graph as a single callable tool, so one agent invokes another synchronously and stays in control
Event Driven	<code>events.wire_event_driven</code>	Subscribes any compiled graph to a pub/sub topic, so it runs automatically when an event arrives — a trigger mechanism layered on top of any of the above, not a topology of its own

Human-in-the-loop is native to the engine: `interrupt()` pauses execution mid-tool-call and persists state via a checkpoint; a human resumes it with `Command(resume={"approved": True/False})`.

3.3 Layer 03 — Harness / Runtime Plane

Where a run actually executes, independent of what it's doing.

- **RunBudget** — per-thread cost and step ceilings, enforced fail-closed (a runaway loop is stopped mid-flight by raising, not silently capped)
- **CircuitBreaker** — opens per tool after repeated consecutive failures, closes on the next success
- **with_retry** / **with_timeout** — retry with exponential backoff; a wall-clock timeout on any blocking call
- **RateLimiter** — a generic token-bucket limiter shared by the LLM Gateway and the Tools Gateway
- **Execution sandbox** (`sandbox.py`) — restricted-namespace, timeout-bound code execution, and the same thing exposed as a registrable tool

Honest limit: this is process-level isolation — a restricted namespace and a timeout, not a container or a VM. It stops accidental misuse, not a determined adversary.

3.4 Layer 04 — Tools Gateway

Everything an agent can do.

- **ToolRegistry** — register any Python callable; RBAC-scoped invocation against a `Policy`
- **MCP client bridge** (`mcp_tools.py`) — connects to any MCP server over stdio or streamable HTTP and registers its tools into the same registry, indistinguishable from local tools to every downstream layer. One registry can compose a local function, multiple independent MCP servers, and other tool sources simultaneously.
- **HTTP/REST adapter** (`http_tools.py`) — wraps any REST endpoint as a tool
- **Data connector** (`data_connectors.py`) — structured data sources (SQL databases, warehouses); SELECT-only, table-allowlisted

- **ToolCache** / **per-tool RateLimiter** — result caching and rate limiting at the tool level
- **Idempotency** — `ToolRegistry.invoke(..., idempotency_key=...)`: a retried call with the same key returns the first call's result instead of re-executing the side effect (a double-refund guard). Only successful results are cached; a failed attempt stays retryable
- **Security** — `ToolManifestRegistry` pins a hash of each tool's signature and description, flagging silent drift; `EgressPolicy` allowlists which hosts a given tool may reach

3.5 Layer 05 — LLM Gateway

Every model, one contract.

The entire contract a provider must satisfy is one method: `complete(messages, *, model, **kw) -> LLMResponse`. Everything else in the framework depends only on that shape, never on a vendor SDK directly.

- **AnthropicProvider** / **OpenAIProvider** — reference implementations
- **MultiProvider** — dispatches by model name to whichever vendor client owns it, so failover is genuinely cross-vendor
- **PromptCache** — exact-match caching, keyed by (model, message sequence)
- **RateLimiter** — shared with the Tools Gateway
- **ModelRegistry** — tracks measured eval scores per model, so routing can be built from evidence rather than a hand-picked list

3.6 Layer 06 — Context Layer

Everything an agent remembers.

- **Working memory** — per-thread scratch state
- **Episodic memory** — full thread history

- **Semantic memory (RAG)** — a `VectorStore` protocol; `InMemoryVectorStore` (keyword-overlap, zero dependencies) and `ChromaVectorStore` (a real embedded vector database, any embedding function) both implement it. Automatically injected into the prompt during `think()`, or upgraded through a full `ContextEngine` pipeline (retrieve → rank → filter → compress → assemble → token-budget).
- **Procedural memory** — learned tool-use patterns: which sequence of calls tends to complete a given task type
- **Knowledge graph** — a `(subject, relation, object)` triple store, *plus real ontology semantics*: named classes, a subclass hierarchy, instance-of typing, and transitive `is_a` queries — not just flat triples
- **Profiles** — per-user/org key-value state

3.7 Left Rail — Identity & Governance

- **Identity** — every agent or sub-agent carries a signed identity, not a shared API key
- **Policy** — what an identity may do (`allowed_tools`), spend (`max_cost_usd_per_thread`), and what requires approval, gated additionally by a **six-level autonomy model** (below)
- **Audit trail** — `AuditLog` (in-memory) or `JSONLAuditLog` (durable, file-streamed); every tool call and approval decision recorded
- **Secrets** — `CredentialVault` (environment-backed) or `VaultCredentialProvider` (a real HashiCorp Vault integration over its KV v2 API)
- **Multi-tenancy** — `Identity.tenant_id` threaded through cost, audit, and policy
- **Escalation** (`escalation.py`) — the third outcome besides auto-approve and deny: `Escalator` hands a case to a human queue or another agent instead of blocking outright. `GuardrailResult.escalate` (default `False`) is how a check signals this rather than a flat denial. Distinct from `requires_approval`, which pauses the *same* thread synchronously via `interrupt()` — an escalation hands the case off entirely, e.g. to an async human queue

AUTONOMY LEVELS

Level	Meaning
L0 — Answer	No action at all; respond only
L1 — Recommend	May suggest an action, never take it
L2 — Draft	May prepare a non-destructive action; destructive ones stay blocked
L3 — Approval (default)	May execute, but tools flagged <code>requires_approval</code> still need a human
L4 — Policy-Bound	May execute anything policy allows, no per-call approval
L5 — Full Autonomy	Same technical ceiling as L4; the distinction is operational trust, not code

3.8 Right Rail — Evaluation, Guardrails & Reinforcement

EVALUATION — FOUR LEVELS

Level	Unit	Method
Atomic	One LLM turn or tool call	Automated assertions, scored inline
Component	One agent/node across a session	Per-node success rate, handoff correctness
Flow	A full multi-step task	Trajectory efficiency, goal accuracy — recorded automatically when a task completes
Overall	The business outcome	CSAT, resolution rate — needs a production feedback wire-up, not built by default

THE KPI FRAMEWORK — COMPREHENSIVE, AND PLUGGABLE

Rather than a fixed set of guardrail categories, `kpi.py` defines a `KPI` (a name, a scoring function, a direction, a threshold, and a weight) and a `KPIBoard` to register any number of them. Ten reference KPIs ship out of the box (efficiency, conciseness, policy adherence, response time, cost, groundedness, user satisfaction, task success, tool error rate, hallucination rate), plus a catalogue of **scoring methods**:

- `llm_judge_kpi` — LLM-as-judge, rates any stated criterion
- `db_match_kpi` — deterministic fact-check against a real database (a caller-supplied lookup, so this module never imports a database concept directly)
- `reference_check_kpi` — grounding against cited documents, keyword-overlap, no model call

These compose freely on one board with whatever weighting a use case needs — for example, correctness 50% (LLM judge) + a database fact-check 30% + document grounding 20%. In testing, the deterministic database check caught a fabricated claim an LLM judge alone missed, which is the actual argument for mixing methods rather than picking one.

GUARDRAILS — FOUR GATES

Gate	Moment	Default mechanism
Input	On the way in	Prompt-injection marker match
Output	On the way out	PII pattern match + redaction
Action	Before it acts	Autonomy level, approval requirement, spend cap
Runtime	While it runs	Per-step timeout, step ceiling, cost ceiling, circuit breaker

Both the guardrail engine and the evaluation harness are formalized behind `Protocol s` (`GuardrailChecks`, `Evaluator`) with a second, genuinely different implementation proven for each: `LLMGuardrails` (LLM-judgment instead of regex — verified catching a paraphrased injection attempt the regex engine missed) and `JSONLEvalSink` (a durable file stream instead of an in-memory harness).

Real policy-as-code is also supported, in two languages behind the same

`PolicyEngine` protocol and the identical convenience input shape: `OPAPolicyEngine` (Rego, queries a running Open Policy Agent server's REST API, verified against a live local instance) and `CedarPolicyEngine` (AWS's Cedar language, via the embedded `cedarpy` engine — no server needed, runs in-process). A team swaps Rego for Cedar without touching call sites.

PLANNING

`planning.py` defines a `DecisionMaker` protocol — anything with `.choose(candidates) -> str`. `Planner` scores candidates against a weighted `KPIBoard` (used to pick the cheapest model meeting a latency bar, for instance). `BanditSelector` is a second, structurally different implementation — an epsilon-greedy multi-armed bandit that learns from observed rewards instead of a fixed scoring function — proving the protocol is a real swap point. `StrategySelector` applies the same idea to *topology* choice: score a request's complexity and risk, and let a rule set pick single-agent vs. supervisor vs. debate/judge automatically, rather than hardcoding one pattern per use case.

REINFORCEMENT LOOP

Five stages, closing the loop at different speeds:

1. **Think** — every LLM call is traced with model, cost, and latency
2. **Act** — tool/LLM calls through the gateways, inside the guardrails
3. **Evaluate** — atomic, component, and flow scores attached automatically
4. **Learn — prompt level** — `PromptOptimizer` curates best-scoring trajectories into few-shot exemplars, same-day
5. **Learn — policy/model level** — `PreferenceStore` turns human approve/deny decisions into chosen/rejected pairs, exportable for DPO or distillation

Honest limit: the data pipeline for stage 5 is built; actually running a fine-tuning or DPO job against that exported data is a separate ML training pipeline, out of scope here.

OPERATIONAL TOOLING — FLAGS, VERSIONS, EXPERIMENTS, BATCH

Four cross-cutting concerns that don't belong to one layer diagram box, because every layer above can use them:

- **Feature flags** (`feature_flags.py`) — `FeatureFlagProvider` protocol; `StaticFeatureFlagProvider` is either a plain on/off switch or a 0-100 rollout percentage, bucketed by a stable hash of `(flag, identity)` so a given identity's answer never flaps between calls
 - **Versioning & rollback** (`versioning.py`) — `VersionStore` protocol; `FileVersionStore` writes every `publish()` as an immutable new version and moves a `current` pointer, so `rollback()` is instant and nothing is ever lost. `VersionedPromptLibrary` is the drop-in, version-backed alternative to `PromptLibrary`
 - **A/B testing** (`experiments.py`) — `Experiment.assign(identity)` deterministically buckets an identity into a variant (stable across calls, proportional to configured weights); `ExperimentTracker` aggregates a recorded metric per `(experiment, variant)` so two prompts, policies, or topologies can be compared with real numbers
 - **Batch & schedule** (`batch.py`) — `run_batch()` runs a compiled graph once per item, concurrently, each on its own thread id — for offline/bulk jobs, distinct from `build_fanout_graph`'s in-turn parallelism. `IntervalScheduler` is a zero-dependency recurring-job runner (a real `threading.Timer` chain); swap in `croniter` behind the same `Scheduler` protocol for real cron syntax
-

4. Security, Observability & Cost

These are not diagram decoration — every layer above reports into all three, on every request.

Security. Signed tool manifests, an egress allowlist per tool, a real Vault-backed secrets provider, and a red-team posture: assume every input is adversarial and every tool call a blast-radius decision. `docs/OWASP_LLM_TOP10.md` cross-checks every OWASP LLM Top 10 risk against specific, tested code — 7 of 10 have direct mitigations; the other 3 are named as open gaps rather than stretched to fit.

Observability. `Tracer` (in-memory, structured spans) or `OTelTracer` (a real OpenTelemetry SDK integration, genuine spans verified) — both behind a `TracerLike` protocol. `Metrics` and `check_alerts` compute latency percentiles, tool error rate, and threshold-based alerting from the trace history.

Cost. Metered per completed task, not reconstructed after the fact:

`CostLedger.close_task()` fires the moment a thread reaches its end, attributing cost to a finished unit of work.

5. The Extensibility Model

Every external system this framework touches is a small `Protocol`, proven with at least two real implementations:

Concern	Protocol	Implementations verified
LLM	<code>Provider</code>	Anthropic, OpenAI
Tools	<code>ToolSpec</code> (any callable)	Function, MCP server, HTTP API, SQL database, sandbox
Vector store	<code>VectorStore</code>	In-memory keyword, ChromaDB
Knowledge graph	<code>KnowledgeGraphStore</code>	In-memory triples + ontology
Planning	<code>DecisionMaker</code>	KPI-weighted scoring, epsilon-greedy bandit
Evaluation	<code>Evaluator</code>	In-memory harness, durable JSONL sink
Guardrails	<code>GuardrailChecks</code>	Regex heuristics, LLM-as-judge
Policy engine	<code>PolicyEngine</code>	Plain dataclass, real OPA/Rego, real Cedar (<code>cedarpy</code> , in-process)
Secrets	<code>SecretsProvider</code>	Environment-backed, real HashiCorp Vault
Audit	<code>AuditSink</code>	In-memory, durable JSONL
Tracing	<code>TracerLike</code>	In-memory JSON, real OpenTelemetry
Event bus	<code>EventBus</code>	In-memory, Kafka (client verified, no broker to round-trip against in this environment)
Agent-to-agent	—	Real A2A protocol, verified end-to-end against a live local server
Messaging channel	—	Real Slack Events API integration
Feature flags	<code>FeatureFlagProvider</code>	Static (bool or percentage-rollout)
Prompt/policy versioning	<code>VersionStore</code>	File-backed, immutable-version + current-pointer
Escalation	<code>Escalator</code>	In-memory ticket queue

Concern	Protocol	Implementations verified
Idempotency	IdempotencyStore	In-memory, TTL-bound
Scheduling	Scheduler	Interval-based (threading.Timer)

Nothing in `orchestration.py` is typed against a concrete implementation of any of these — every caller only ever depends on the protocol’s one or two methods.

6. Deployment

No layer in `agent_foundry/` imports a cloud-specific SDK. The portability boundary is the container: `serve.py` wraps any compiled graph as a FastAPI app, and the repository's `Dockerfile` packages it. The same image runs on AWS ECS/Fargate/App Runner, GCP Cloud Run, Azure Container Apps, any Kubernetes cluster, or a bare VM.

7. Module Reference

Module	Layer	Responsibility
<code>contracts.py</code>	Core	Identity, Policy, AutonomyLevel, AgentRole, ToolSpec, LLMResponse, GuardrailResult, EvalRecord
<code>orchestration.py</code>	Orchestration	The three primitives, seven+ topologies
<code>llm_gateway.py</code>	LLM Gateway	Providers, routing, cache, rate limiting, model registry
<code>tools_gateway.py</code>	Tools Gateway	Registry, RBAC, caching, rate limiting
<code>mcp_tools.py</code>	Tools Gateway	MCP client bridge
<code>http_tools.py</code>	Tools Gateway	REST API adapter
<code>data_connectors.py</code>	Tools Gateway	SQL/warehouse connector
<code>sandbox.py</code>	Runtime	Restricted code execution
<code>context.py</code>	Context	Memory, RAG, knowledge graph/ontology, context engine
<code>blackboard.py</code>	Orchestration	Shared-workspace primitive
<code>prompts.py</code>	—	Prompt file loading
<code>guardrails.py</code>	Guardrails	Regex and LLM-judgment engines
<code>eval.py</code>	Evaluation	In-memory and durable harnesses
<code>kpi.py</code>	Evaluation	KPI/KPIBoard, scoring method catalogue
<code>planning.py</code>	Planning	DecisionMaker, Planner, BanditSelector, StrategySelector

Module	Layer	Responsibility
<code>policy_engine.py</code>	Security	Real OPA and Cedar integrations
<code>security.py</code>	Security	Manifests, egress, audit, secrets
<code>escalation.py</code>	Governance	Escalation tickets, distinct from deny/approve
<code>events.py</code>	Integration	Pub/sub, event-driven pattern
<code>a2a_bridge.py</code>	Integration	Agent-to-agent protocol
<code>autogen_bridge.py</code>	Integration	Cross-framework interop (Microsoft AutoGen)
<code>channels.py</code>	UI/UX	Slack and webhook channels
<code>observability.py</code>	Observability	Tracing, metrics, cost ledger
<code>reinforcement.py</code>	Reinforcement	Exemplar curation, preference export
<code>runtime.py</code>	Runtime	Budgets, retries, circuit breaker, rate limiter
<code>quickstart.py</code>	Quickstart	The plug-and-play LangChain path
<code>scaffold.py</code>	Tooling	Generates a new agent's starting files
<code>serve.py</code>	Deployment	HTTP serving
<code>ui/console.py</code>	UI/UX	CLI conversational surface, HITL console
<code>feature_flags.py</code>	Operational	On/off and percentage-rollout flags
<code>versioning.py</code>	Operational	Prompt/policy versioning and rollback
<code>experiments.py</code>	Operational	A/B variant assignment and tracking
<code>batch.py</code>	Operational	Bulk batch runner, interval scheduler

This document describes the architecture. See the companion implementation guide for how to build a specific agent on top of it.